UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Algoritmos e Estruturas de Dados

A • Exame Época Recurso • 27 de janeiro de 2025 • 15:00 – 17:00

- Para perguntas com resposta de escolha múltipla, respostas erradas com cotação c e n respostas possíveis descontam $-c / (n - 1)$.
- As respostas às perguntas de desenvolvimento deverão ser as mais detalhadas possíveis, indicando os vários passos intermédios da resolução.
- Na mesa em que está a fazer o exame deve ter apenas canetas, identificação, este enunciado, e folha de teste.

Grupo I: Perguntas de escolha múltipla

1. (0.5) Depois de ter implementado um determinado algoritmo, decidiu executar ensaios de razão dobrada para tentar estimar de forma empírica a complexidade temporal assimptótica do algoritmo. Os resultados obtidos foram os seguintes:

i	n	time(ms)	estimated r
…			
8	12800	005,7292	0,000
9	25600	007,8125	1,364
10	51200	016,6667	2,133
11	102400	028,6458	1,719
12	204800	085,4167	2,982
13	409600	228,6458	2,677
14	819200	465,6250	2,036

Indique a complexidade temporal estimada, tendo em conta os resultados obtidos:

a) $O(n^3)$
b) $O(\log n)$
c) $O(1)$
d) $O(n)$
e) $O(n^2)$

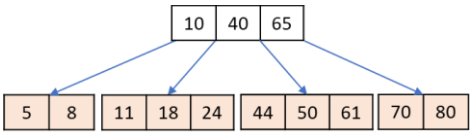
2. (0.5) Qual algoritmo de ordenação é mais eficiente para ordenar um *array* já quase ordenado?

a) BubbleSort
b) QuickSort
c) InsertionSort
d) MergeSort
e) Heapsort

3. (0.5) Considere o método *partition* aplicado ao seguinte *array* [4, 8, 5, 0, 1, 7, 3, 2], com low=0, high=7, e com escolha do pivô como o elemento mais à esquerda. Indique qual das seguintes sequências corresponde aos vários estados do *array*, no fim de cada troca feita pelo método.

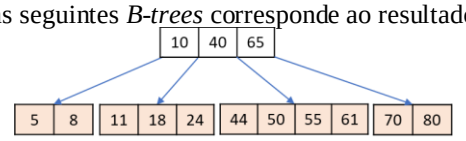
a) [4,2,5,0,1,7,3,8][4,2,3,0,1,7,5,8][1,2,3,0,4,7,5,8]
b) [4,2,5,0,1,7,3,8][4,2,3,0,1,7,5,8]
c) [4,0,2,5,1,3,7,8][4,0,2,3,1,5,7,8][1,0,2,3,4,5,7,8]
d) [4,2,5,0,1,7,3,8][1,2,5,0,4,7,3,8]
e) [4,2,5,0,1,7,3,8][4,2,7,0,1,5,3,8][1,2,7,0,4,5,3,8]

4. (0.5) Considere a seguinte *B-tree* (designada de *FatTree* no projeto P3), de grau mínimo ***g=2***:

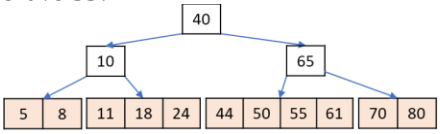


Qual das seguintes *B-trees* corresponde ao resultado de se inserir a chave **55**?

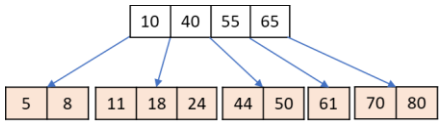
a)



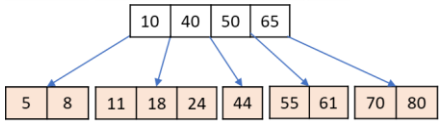
b)



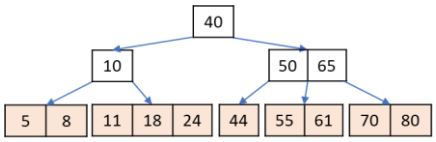
c)



d)



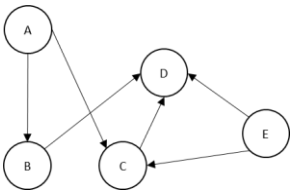
e)



5. (0.5) Considere uma matriz esparsa, com m linhas, n colunas, e k elementos não nulos. Usando uma representação da matriz esparsa com uma tabela de dispersão, tal como implementado no Projeto P4, qual é a complexidade temporal assimptótica para obter a soma de todos os elementos da matriz.

a) $O(m \times n)$
b) $O(k)$
c) $O(m+n)$
d) $O(\log k)$
e) $O(k \log m \times n)$

6. (0.5) Para o seguinte grafo dirigido



Indique qual das seguintes ordenações é uma ordenação topológica do grafo

a) A, B, C, D, E
b) B, C, D, A, E
c) E, A, B, C, D
d) A, B, E, D, C
e) Não existe nenhuma ordenação topológica válida

Grupo II: Perguntas de desenvolvimento

7. (2.5) Exercícios sobre a linguagem de programação Java.

a) (1.0) Escreva um método estático chamado `isPrime`, que recebe um número inteiro positivo `n`, e verifica se o número é primo, retornando um booleano (verdadeiro caso seja primo e falso caso contrário). Um inteiro positivo é primo se tiver 2 e apenas 2 divisores, o próprio número e o número 1. Embora não seja a forma mais eficiente de o fazer, para determinar se um número *n* é primo, podemos testar todos os inteiros no intervalo `]1,n[`, e se nenhum deles for divisor de p, então *n* é primo. Atenção que o número 1 não é considerado primo porque só tem um divisor. Exemplos: `isPrime(1)` retorna falso, `isPrime(8)` retorna falso e `isPrime(11)` retorna verdade.

b) (1.5) Escreva um método estático de ordem superior, designado `todos`, que recebe um *array* de inteiros `a`, e um predicado do tipo inteiro `(Predicate<Integer>) p`, e que retorna um booleano. Este método vai percorrer todos os elementos do *array* testando-os com o predicado p. Se todos os elementos satisfizerem o predicado p, o método retorna verdade, e em caso contrário retorna falso. Caso o *array* esteja vazio deve ser retornado verdade. Por exemplo, ao executar a instrução `todos([7,3,2,0,8,9], (i)->i>=0)` é retornado verdadeiro.

Dica: para executar a interface funcional `Predicate` usa-se o método `test`.

8. (1.5) Considere o seguinte código para a implementação do método *quickSelect*, implementado no Projeto P3.

```
private static <T extends Comparable<T>> T quickSelect(T[] a, int i, int low, int high)
{
    if (high <= low) return a[low];
    int j = partition(a, low, high);
    if(j == i) return a[i];
```

```
    if(i < j) return quickSelect(a,i,low,j-1);
    else return quickSelect(a,i,j+1,high);
}
```

Sabendo que a complexidade temporal do método *partition* para um *subarray* de tamanho *n* é *n*, e ignorando a condição de *early exit*, calcule uma aproximação tilde para a ordem de crescimento **no melhor caso** do método *quickSelect* em função do tamanho do *subarray* original *n*. O tamanho de um subarray *n* corresponde a `(high - low)+1`. Lembre-se que o método *partition* é o usado no algoritmo *QuickSort*, e que o melhor caso acontece quando o *partition* parte o *subarray* ao meio, e que a soma dos *n* primeiros termos de uma série geométrica é dada por $s_n = \frac{a_1(1-r^n)}{1-r}$.

9. (1.0) Considere a seguinte fila, em que o início da fila contém o número 0.

6	2	8	9	0

Desenhe o estado da fila resultante das seguintes operações:

dequeue (), dequeue(), x=dequeue(), enqueue(3), enqueue(5), enqueue(x), dequeue()

10. (1.0) O João implementou a seguinte lista ligada:

```
public class LinkedList<T> implements IList<T> {
    private class Node  {
        T item;
        Node next;
    }
    private Node first;

    public LinkedList() {
        this.first = null;
    }

    public void add(T item) {
        Node newNode = new Node();
        newNode.item = item;
        newNode.next = this.first;
        this.first = newNode;
    }
```

Uma lista ligada corresponde a uma sequência de nós ligados, até chegar a um nó que não aponta para nada. O membro first é um ponteiro para o 1.º nó da lista ligada, inicialmente nulo. O João esqueceu-se usar um membro adicional para guardar o tamanho da lista. No entanto, embora não seja eficiente, é possível determinar o tamanho de uma lista ligada mesmo assim. Implemente em Java o método size() para a classe LinkedList que não tem argumentos e que devolve o tamanho da lista, sem adicionar nenhum membro ou campo adicional à classe (poderá obviamente usar variáveis locais no método).

11 (2.0) Considere o algoritmo de ordenação *shellsort* usando a sequência de *Knuth*, para ordenar o seguinte *array*:

a=[5, 7, 2, 10, 8, 4, 9, 3, 1, 8, 6, 1, 1, 2, 3]

Indique o estado do *array* após cada operação de *h-sorting*, assumindo que o valor de *h* inicial é 13.

12. (1.5) Qual a principal vantagem de uma *B-tree* (designada por *FatTree* no Projeto P3), face a uma árvore binária de pesquisa. Que características da implementação de uma *B-tree* ajudam a garantir esta vantagem?

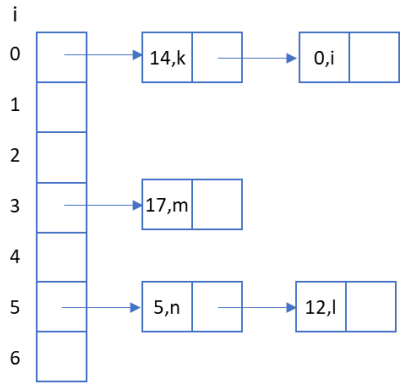
13. (2.5) Resolva os seguintes exercícios sobre amontoados (heaps).

a) (1.5) Desenhe a representação em árvore do *max-heap* obtido quando se insere os seguintes elementos num *heap* inicialmente vazio: 7, 2, 5, 0, 1, 6, 3, 8. Desenhe uma nova árvore sempre que um novo elemento for trazido para a raiz da árvore.

b) (1.0) Indique, justificando, se o seguinte *array* pode ser considerado um *max-heap* (assumindo que a raiz do *heap* começa na posição 1 do array).

	9	8	3	7	6	1	2

14. (2.5) Considere a seguinte tabela de dispersão com resolução de colisões por encadeamento separado (separate chainning) de dimensão $m = 7$, e com função de dispersão $h_1(k) = k \bmod 7$.

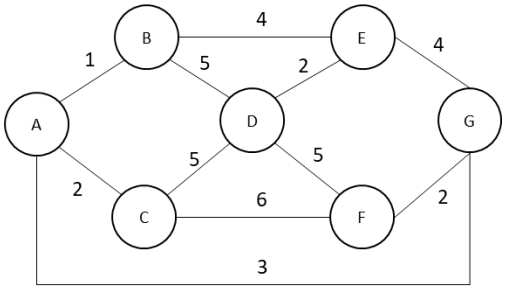


A tabela foi inicialmente preenchida com os pares <*chave*,*valor*> pela seguinte ordem: <*0*,*i*>, <*5*,*n*>, <*12*,*l*>, <*14*,*k*> e <*17*,*m*>. Uma célula vazia representa uma lista vazia ou um ponteiro *null*.

a) (1.0) Desenhe a tabela que se obtém apagando as chaves 14 e 17.

b) (1.5) Desenhe a tabela que se obtém, após inserir os seguintes pares <*chave*,*valor*> na **tabela original** pela ordem especificada. Indique o valor de h1 calculado para cada chave.
<*3*,*a*>,<*7*,*b*>,<*2*,*c*>,<*16*,*d*>,<*21*,*e*>

15. (2.5) Considere o grafo dirigido da figura seguinte:



Considere a execução do algoritmo de Prim sobre o grafo pesado não dirigido da figura abaixo. Qual a sequência pela qual o algoritmo de Prim visita as arestas de corte, assumindo que o algoritmo inicia a procura a partir do **vértice A**? (Em caso de empate, utilize a ordem alfabética do menor vértice de um arco para desempatar). Desenhe a árvore de cobertura mínima obtida pelo algoritmo.